# Classification: Logistic regression

# Classification

- In contrast to the regression problem where we would like to predict the value of a quantitative variable, we will now consider prediction of qualitative variables, i.e. which class an observation belongs to.
- $$Y = \begin{cases} 1 \text{ with probability } p_1 \\ 2 \text{ with probability } p_2 \\ \vdots \\ K \text{ with probability } p_K \end{cases}$$
- where $p_1 + p_2 + ... + p_K = 1$
- We want to predict $Y_i$ based on $\mathbf{X}_i$
- Models for this are usually based on estimating $p_k$ as a function of $\mathbf{X}_i$

$$p_k = P(Y_i = k | \mathbf{X}_i = \mathbf{x}_i) = f(\mathbf{x}_i)$$

# A natural quality measure of predictions

- Data: $\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_n, \mathbf{x}_{n+1}, ...\mathbf{x}_{n+m}$
  $y_1, y_2, ..., y_n, y_{n+1}, ...y_{n+m}$
- $$\hat{y}_i = \text{Prediction of } y_i$$
- Training error rate:

$$trainER = \frac{1}{n} \sum_{i=1}^{n} I(\hat{y}_i \neq y_i)$$

- Test error rate:

$$testER = \frac{1}{m} \sum_{i=n+1}^{n+m} I(\hat{y}_i \neq y_i)$$

- $I(x)$ is the indicator function (one if x true, zero otherwise)

# A first example

```
import numpy as np
import pandas as pd
from ISLP import load_data
Default = load_data("Default")
Default.describe(include = "all").round(2)
```

| | default | student | balance | income |
|---|---|---|---|---|
| count | 10000 | 10000 | 10000.00 | 10000.00 |
| unique | 2 | 2 | NaN | NaN |
| top | No | No | NaN | NaN |
| freq | 9667 | 7056 | NaN | NaN |
| mean | NaN | NaN | 835.37 | 33516.98 |
| std | NaN | NaN | 483.71 | 13336.64 |
| min | NaN | NaN | 0.00 | 771.97 |
| 25% | NaN | NaN | 481.73 | 21340.46 |
| 50% | NaN | NaN | 823.64 | 34552.64 |
| 75% | NaN | NaN | 1166.31 | 43807.73 |
| max | NaN | NaN | 2654.32 | 73554.23 |

# A first example

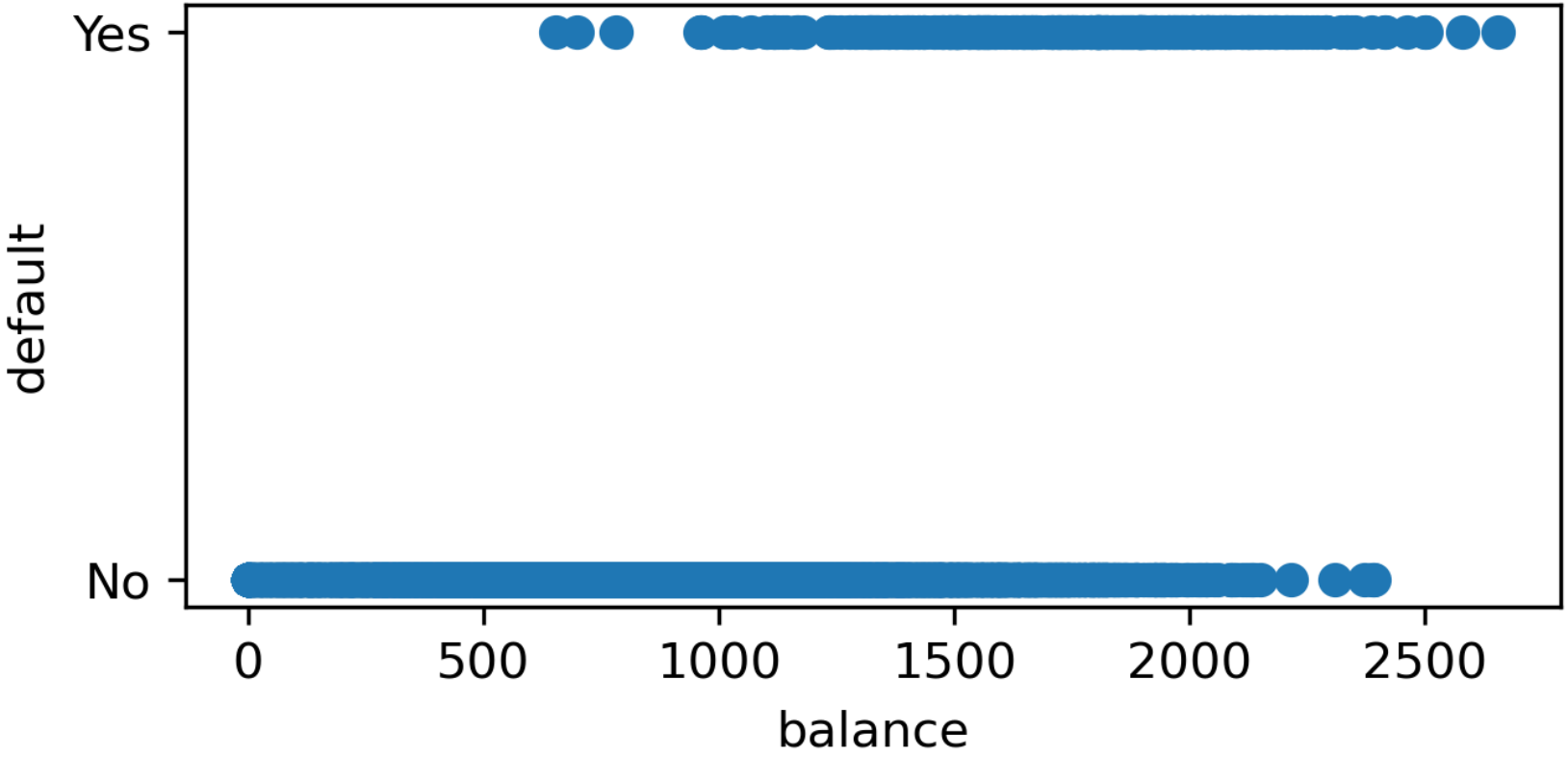
```
pd.crosstab(Default.student, Default.default,
    normalize='index').round(3)
```

| default<br>student | No | Yes |
|---|---|---|
| No | 0.971 | 0.029 |
| Yes | 0.957 | 0.043 |

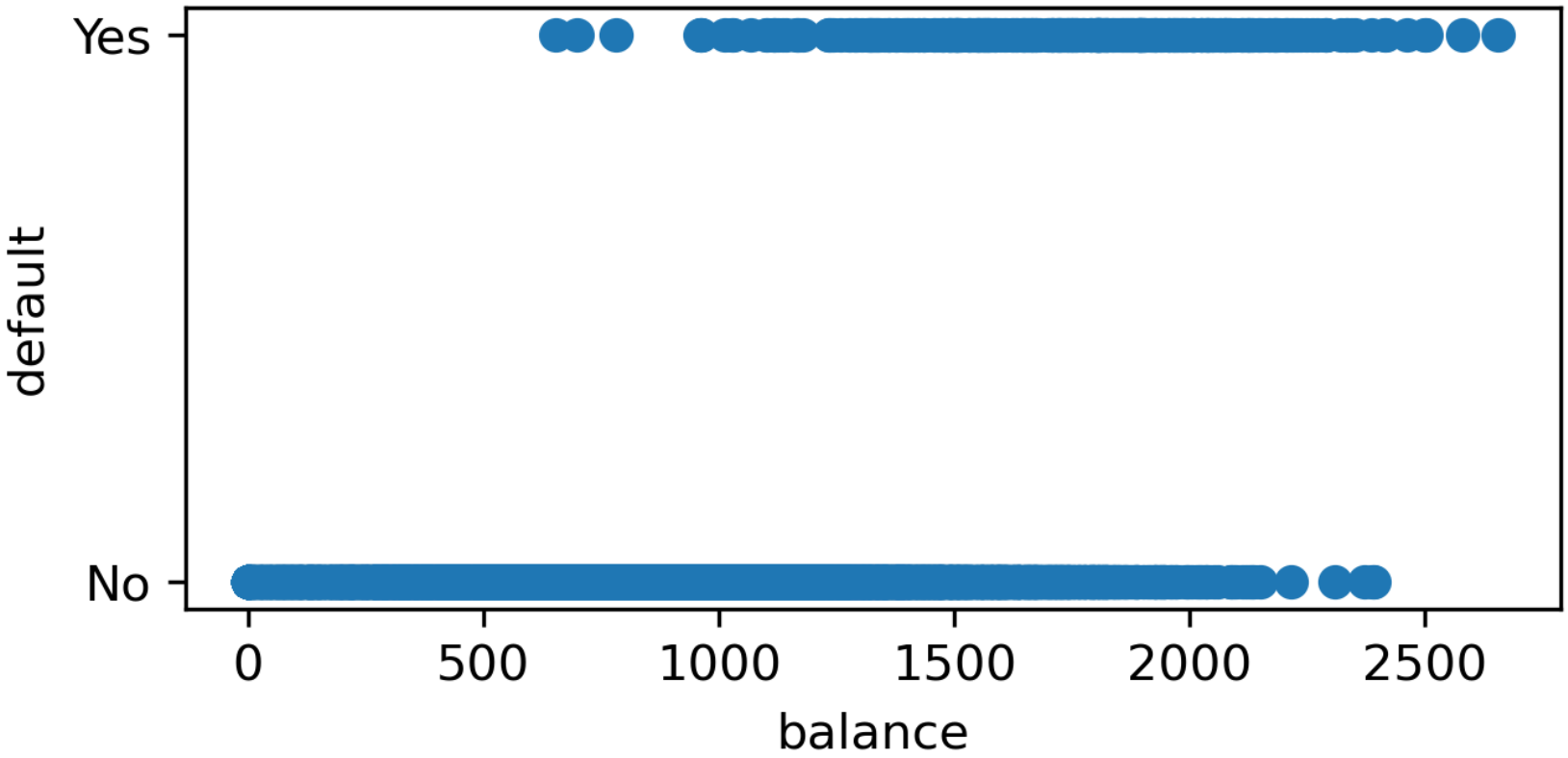
- $$P(Y_i = \text{Yes} | X_i = \text{Yes}) > P(Y_i = \text{Yes} | X_i = \text{No})$$

# Account balance against default variable

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(5, 2.2))
ax.scatter(Default.balance,Default.default)
plt.xlabel("balance")
plt.ylabel("default")
plt.show()
```



# Our first classification method - a linear probability model



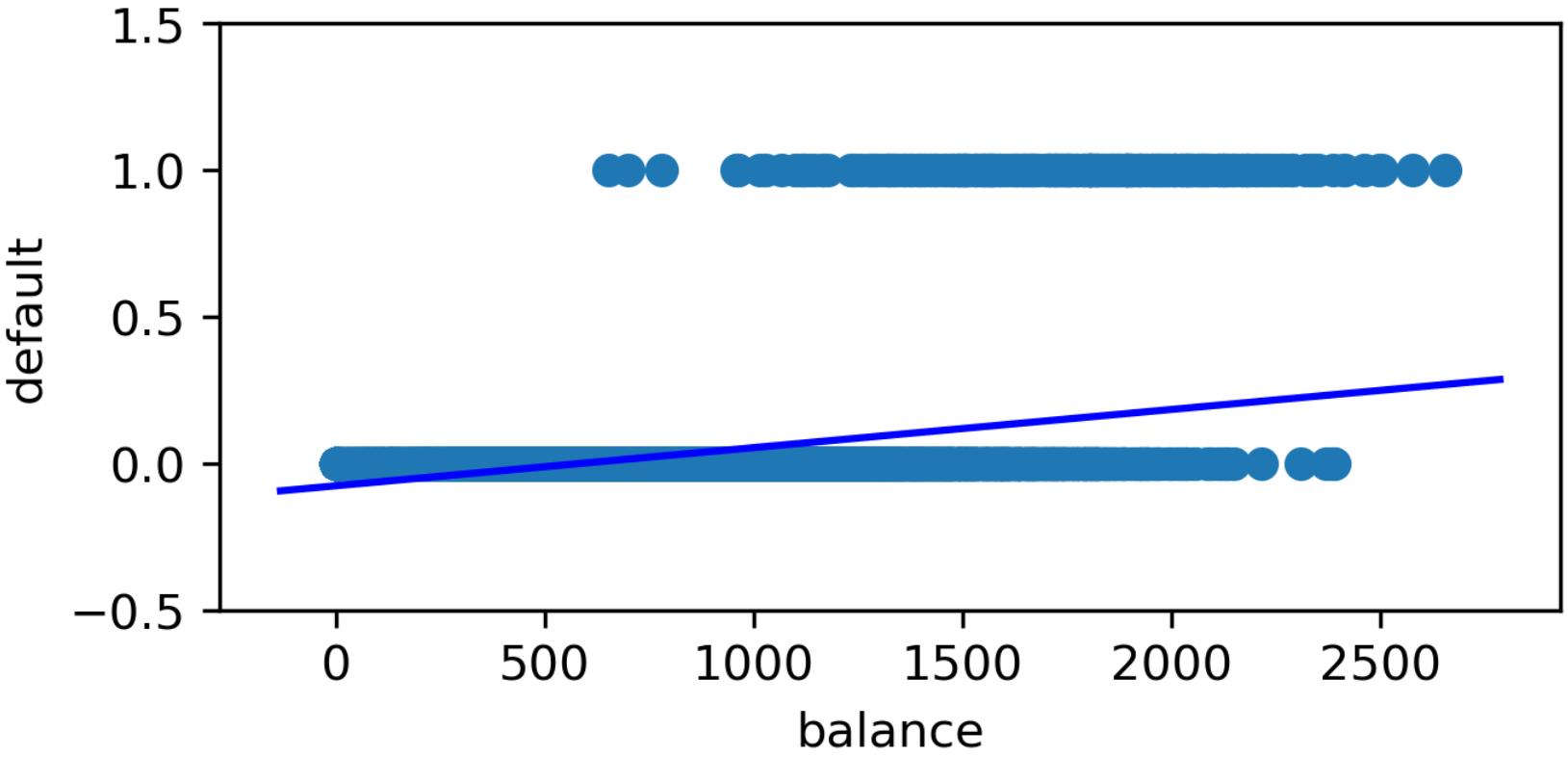

- We need to make `default` a numerical value
- $Y_i = 1$ if default and $0$ otherwise

# Our first classification method - a linear probability model

```
import statsmodels.formula.api as smf
from ISLP.models import summarize
Default["Y"] = Default["default"].map({"Yes": 1, "No": 0})
linprob = smf.ols('Y ~ balance', data=Default).fit()
summarize(linprob)
```

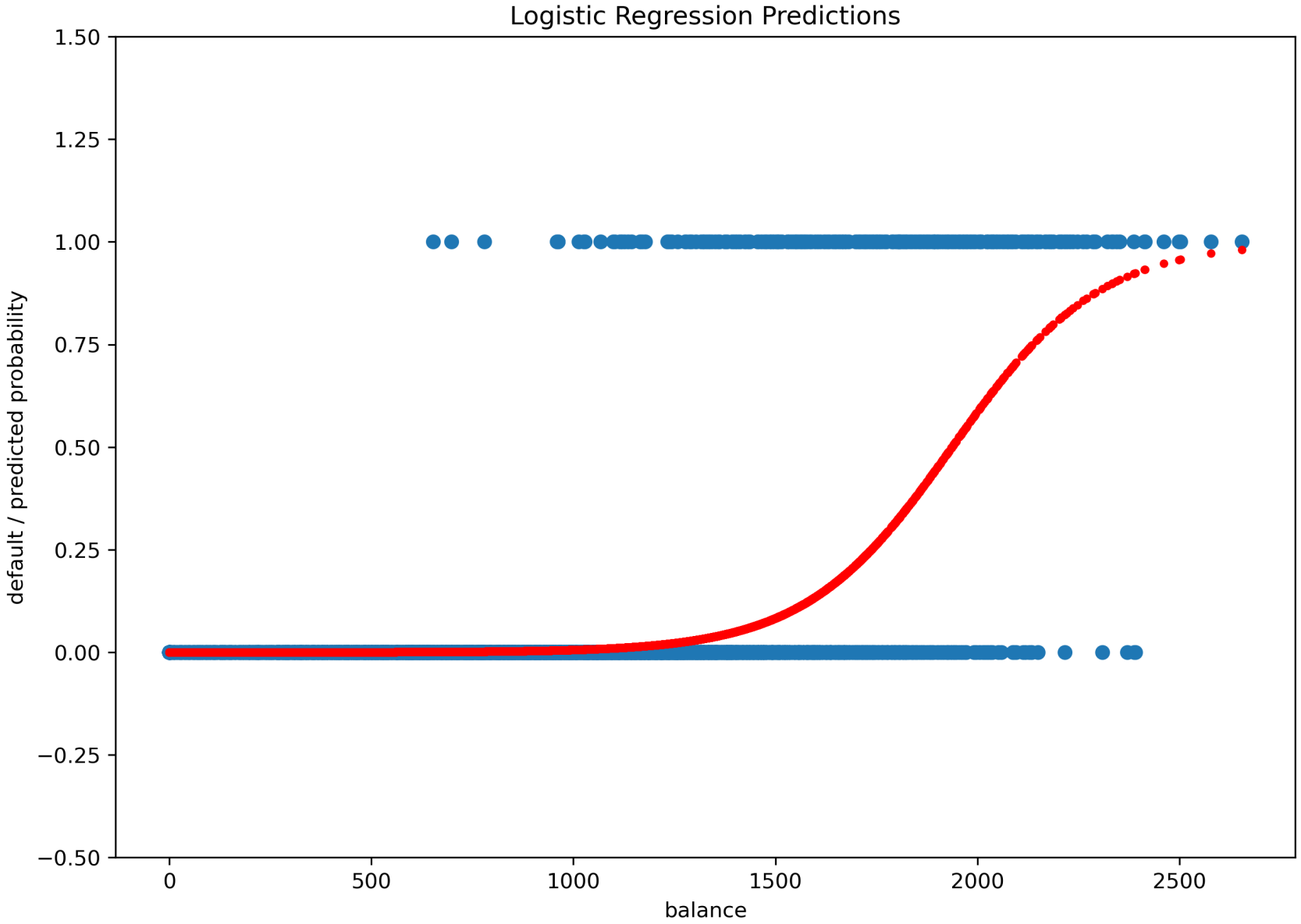
| | coef | std err | t | P>\|t\| |
|---|---|---|---|---|
| Intercept | -0.0752 | 0.003000 | -22.416 | 0.0 |
| balance | 0.0001 | 0.000003 | 37.374 | 0.0 |

# Our first classification method - a linear probability model




- Any obvious problems?

# Our first proper classification method - logistic regression




- Idea for prediction: Predict default when the estimated

# Logistic regression

The prediction of $Y$ is done in two steps

1. Estimate the probability of $Y = 1$ given an observed value of $X$, $X = x$
2. Use the estimated probability to decide whether $Y$ should be predicted as one or zero

- $$P(Y = 1|X = x) = p(x) = \frac{e^{\beta_0+\beta_1 x}}{1 + e^{\beta_0+\beta_1 x}} = \frac{1}{1 + e^{-\beta_0-\beta_1 x}}$$

- Note that $\frac{p(x)}{1-p(x)} = e^{\beta_0+\beta_1 x}$ is the *odds*, i.e. the probability of an event occuring in relation to the probability of the event not occuring.

This means that $e^{\beta_1}$ represents the change in relative probability when $x$ increases with one unit.

```
logprob=smf.logit("Y~student",data=Default).fit(disp=False)
summarize(logprob)
```

| | coef | std err | z | P>\|z\| |
|---|---|---|---|---|
| Intercept | -3.5041 | 0.071 | -49.554 | 0.0 |
| student[T.Yes] | 0.4049 | 0.115 | 3.520 | 0.0 |

- $\hat{p}(\text{not student}) = \frac{1}{1+e^{3.5041}} \approx 0.029$
- $e^{0.4049} = 1.50$
- Interpretation: The odds-ratio for default is $50\%$ higher for students $(0.029)$ than for non-students $(0.044)$.

# Maximum likelihood (ML) estimation

Example

$$Y = \begin{cases} 1 \text{ if a customer buys a product} \\ 0 \text{ if not} \end{cases}$$

The probability function for $Y$ is

$$p(y) = P(Y = y) = \begin{cases} p \text{ if } y = 1 \\ 1 - p \text{ if } y = 0 \end{cases}$$

# ML estimation of $p$

- Random sample: $Y_1, Y_2, ..., Y_n$, a sequence of independent random variables
- Observations (data): $y_1, y_2, ..., y_n$, a sequence of numbers
- In this example $m$ of the observations are one and $n - m$ are zero
- The probability of observing the specific numbers $y_1, y_2, ..., y_n$ is

$$\begin{aligned} &P(Y_1 = y_1, Y_2 = y_2, ..., Y_n = y_n) = [\text{indep}] \\ &= P(Y_1 = y_1)P(Y_2 = y_2) \cdots P(Y_n = y_n) \\ &= p^m (1-p)^{n-m} \end{aligned}$$

- The <u>likelihood function</u> $L(p) = p^m (1-p)^{n-m}$
- <u>The maximum likelihood principle</u>: Find the value of $p$ that maximizes the likelihood of observerving the particular values of $Y$ that we have actually observed.

Often easier to maximize the <u>log-likelihood function</u>, which in our example is

$$\ln L(p) = m \ln p + (n - m) \ln(1 - p)$$

- $$\frac{\partial \ln L}{\partial p} = \frac{m}{p} - \frac{n - m}{1 - p} = 0$$
  $$\iff$$
  $$m(1 - p) = p(n - m)$$
  $$\iff$$
  $$\hat{p}_{ML} = \frac{m}{n}$$

- Check that second-order derivative $< 0$.

# ML estimation of parameters in logistic regression

Data: $x_1, x_2, ..., x_n$
$y_1, y_2, ..., y_n$

- Probability of $Y = y$ now a function of $x$.
- $$p(y|x) = \begin{cases} \frac{e^{\beta_0+\beta_1 x}}{1+e^{\beta_0+\beta_1 x}} & \text{if } y = 1 \\ \frac{1}{1+e^{\beta_0+\beta_1 x}} & \text{if } y = 0 \end{cases}$$
- Likelihood function:
  $L(\beta_0, \beta_1) = p(y_1|x_1)p(y_2|x_2)\cdots p(y_n|x_n)$

# ML estimation of parameters in logistic regression

- Can be shown that the log-likelihood function is

$$\ln L(\beta_0, \beta_1) = \sum_{i=1}^{n} \left[ y_i \ln \left( \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}} \right) + (1 - y_i) \ln \left( \frac{1}{1 + e^{\beta_0 + \beta_1 x}} \right) \right]$$

- Must be maximized numerically

# Evaluate accuracy

- Confusion matrix

| | | Prediction | |
|---|---|---|---|
| | | Negative | Positive |
| Actual | Negative | True negative | False positive |
| | Positive | False negative | True positive |

# Default example

```
# Fit logistic regression
logprob = smf.logit('Y ~ student + balance + income',
data=Default).fit(disp=False)
# Estimate probabilities
pred_probs = logprob.predict(Default)
# Convert probabilities to boolean predictions with
# threshold 0.5
pred = pred_probs > 0.5
# Create a confusion table (cross-tabulation)
confusion_table = pd.crosstab(Default['default'],
 pred, rownames=['Actual'], colnames=['Predicted'])
print(confusion_table)
```

```
Predicted  False  True
Actual
No          9627    40
Yes          228   105
```

# Default example

| | | Prediction | |
|---|---|---|---|
| | | Not default | Default |
| Actual | Not default | 9627 | 40 |
| | Default | 228 | 105 |

# Row percentages

```
confusion_table = pd.crosstab(Default['default'],
 pred, rownames=['Actual'], colnames=['Predicted'],
 normalize='index')

print(confusion_table)
```

```
Predicted     False     True
Actual
No         0.995862  0.004138
Yes        0.684685  0.315315
```

# Evaluating the predictions on test data

```
# Test / training data
np.random.seed(123)
n = len(Default)
ntrain = n // 2
ind = np.random.choice(n, size=ntrain, replace=False)
train = Default.iloc[ind]
test = Default.drop(ind)

# Fit logistic regression with logit
logit_model = smf.logit('Y ~ student + balance + income', data=train).fit(disp=False)

# Predict probabilities on test set
pred_probs = logit_model.predict(test)

# Convert probabilities to binary predictions (threshold 0.5)
pred = pred_probs > 0.5

# Confusion matrix
conf_matrix = pd.crosstab(test['default'], pred, rownames=['Actual'], colnames=['Predicted'])
print(conf_matrix)

# Accuracy
accuracy = (pred == test['Y']).mean()
print(f'Accuracy: {accuracy:.4f}')
```

```
Predicted  False  True
Actual
No          4803    19
Yes          119    59
Accuracy: 0.9724
```

# Alternative prediction

- By classifying all $5000$ testobservations as "Not default" we would get an overall prediction accuracy of $4822/5000 \approx 0.9644$.
- Overall prediction accuracy not a good measure when we have one class which is much larger than the other.